Trabajo Práctico II

SQL Avanzado: Funciones, Triggers, WITH, WITH RECURSIVE y Funciones de Ventana

Objetivos

El objetivo de este trabajo práctico es profundizar el uso de SQL mediante mecanismos que permiten incorporar lógica a la base de datos, definir controles de integridad, mantener información de auditoría y resolver consultas complejas sobre estructuras jerárquicas y grafos.

Se trabajará especialmente con funciones almacenadas, triggers, expresiones de tabla comunes (`WITH`), expresiones de tabla comunes recursivas (`WITH RECURSIVE`), consultas sobre árboles y grafos y funciones de ventana.

Los ejercicios deberán resolverse utilizando PostgreSQL y la base de datos correspondiente al Trabajo Práctico II disponible en el Campus Virtual.

1. Control de integridad de las inscripciones mediante triggers

Utilizando la base de datos del Trabajo Práctico II, implementar mediante funciones y triggers los controles necesarios para garantizar las siguientes reglas:

a) Al dar de alta una inscripción, el atributo `estadocursada` debe establecerse siempre en `REGULAR`, independientemente del valor que pudiera especificarse en la sentencia `INSERT`.

b) Al modificar una inscripción y establecer su estado en `PROMOCION`, deberá verificarse que la cantidad de inasistencias sea menor o igual que el valor de `horaspermpromocion` correspondiente al curso en el que se encuentra inscripto el alumno. Si esta condición no se cumple, la modificación deberá ser rechazada.

c) Cuando la cantidad de inasistencias supere el valor de `horaspermregular` correspondiente al curso, el estado de la inscripción deberá cambiar automáticamente a `LIBRE`.

Analice qué eventos debe controlar cada trigger (`INSERT`, `UPDATE`) y si resulta más conveniente utilizar triggers `BEFORE` o `AFTER`.

2. Auditoría de modificaciones

Por razones de auditoría, se requiere mantener un historial de los cambios realizados sobre las inscripciones.

Crear una tabla `log_inscripto` que permita almacenar los datos correspondientes a cada modificación de una inscripción. La tabla deberá contener los atributos necesarios para conservar el estado de la inscripción y, adicionalmente:

a) `fechahora`, de tipo `TIMESTAMP`, cuyo valor predeterminado sea la fecha y hora del sistema.

b) `usuario`, que indique el usuario de la base de datos que realizó la modificación.

Implementar un trigger que registre automáticamente las modificaciones realizadas sobre la tabla `inscripto`.

El registro de auditoría deberá permitir conocer el estado de la inscripción antes y después de la modificación.

Analice qué información proporcionan las variables especiales `OLD` y `NEW` dentro de una función de trigger.

3. Historial de cambios de precios

Dadas las siguientes tablas:

```
productos(id, nombre, precio)

historial_precios(id_producto, precio_antiguo, precio_nuevo, fecha)
```

Crear una función y un trigger que registren en `historial_precios` cada modificación realizada sobre el precio de un producto.

El historial deberá generarse únicamente cuando el precio efectivamente haya cambiado.

Compruebe el funcionamiento realizando modificaciones tanto sobre el precio como sobre otros atributos de un producto.

4. Alumnos con menor cantidad de inasistencias

Utilizando `WITH`, determinar el o los alumnos que hayan acumulado la **menor cantidad total de horas de inasistencia durante el año 2025**.

La consulta deberá devolver, como mínimo:

- identificador del alumno;
- apellido y nombre;
- cantidad total de horas de inasistencia.

La consulta deberá contemplar la posibilidad de que varios alumnos compartan el mínimo.

5. Salarios y desviación estándar

Utilizando `WITH`, calcular el salario medio y la desviación estándar de los salarios de los docentes.

A partir de esos valores, listar los docentes cuyo salario se encuentre dentro del intervalo:

media − desviación estándar ≤ salario ≤ media + desviación estándar

El resultado deberá incluir los datos identificatorios del docente, su salario, la media y la desviación estándar calculadas.

6. Comparación de valores mediante funciones de ventana

Utilizando funciones de ventana, obtener para cada docente:

- identificador;
- apellido y nombre;
- salario;
- salario medio de todos los docentes;
- diferencia entre su salario y el salario medio;
- posición que ocupa según su salario, ordenando de mayor a menor.

Resolver el ejercicio sin utilizar subconsultas para calcular individualmente cada uno de estos valores.

7. Rutas en un grafo

Crear la siguiente tabla:

```
contactos(nodo_origen, nodo_destino, distancia)
```

Cargar datos suficientes para formar un grafo que contenga múltiples caminos entre algunos nodos y al menos un ciclo.

Utilizando `WITH RECURSIVE`, obtener **todos los caminos posibles desde el nodo `A` hasta el nodo `D`**, indicando para cada uno:

- secuencia de nodos recorridos;
- cantidad de aristas recorridas;
- distancia total acumulada.

La consulta deberá impedir que un nodo ya visitado dentro de un camino vuelva a ser recorrido, evitando de esta manera ciclos infinitos.

A partir de la consulta anterior, determinar cuál es el camino de menor distancia entre `A` y `D`.

8. Árbol de dependencias jerárquicas

Utilizando las tablas de personas y dependencias definidas en la base de datos del Trabajo Práctico II, cargar la estructura jerárquica indicada en la figura proporcionada por la cátedra.

Cada persona puede depender directamente de **una sola persona**, por lo que la estructura resultante constituye un árbol.

Mediante una consulta con `WITH RECURSIVE`, obtener para cada persona:

- `idPersona`;
- `dependeDe`;
- `nivel`.

El nivel deberá interpretarse de la siguiente manera:

- Nivel 0: persona ubicada en la raíz de la jerarquía;
- Nivel 1: personas que dependen directamente de la raíz;
- Nivel 2: personas que dependen de las del nivel anterior;
- y así sucesivamente.

Posteriormente, modificar la consulta para mostrar:

- identificador, apellido y nombre de la persona;
- identificador, apellido y nombre de la persona de la cual depende directamente;
- nivel dentro de la jerarquía.

La persona ubicada en la raíz deberá aparecer igualmente en el resultado, aunque no dependa de ninguna otra.

9. Función de dependencia entre personas

Escribir una función que reciba los identificadores de dos personas `X` e `Y` y devuelva un valor booleano indicando si **X depende directa o indirectamente de Y**.

La búsqueda de la dependencia deberá realizarse utilizando una CTE recursiva dentro de la función.

Analice los resultados para los siguientes casos:

- X depende directamente de Y;
- X depende indirectamente de Y;
- X no depende de Y;
- X e Y representan a la misma persona.

10. Relaciones entre personas

Definir un esquema que permita representar relaciones entre personas mediante, como mínimo, las siguientes tablas:

```
personas(...)
```

```
relaciones(...)
```

La estructura deberá permitir representar distintos tipos de relaciones, por ejemplo: padre/madre–hijo/hija, hermano/a, amigo/a, conocido/a y cónyuge.

Debe tenerse en cuenta que la estructura resultante constituye un **grafo general**, por lo que puede contener múltiples caminos entre dos personas y también ciclos.

10.1. Distancia de amistad

Escribir una consulta que, dadas dos personas, determine su **distancia de amistad**.

Se considerará que:

- dos amigos directos tienen distancia 1;
- el amigo de un amigo se encuentra a distancia 2;
- y así sucesivamente;
- si no existe ningún camino formado por relaciones de amistad entre ambas personas, la distancia será 0.

Cuando existan varios caminos posibles, deberá devolverse la **menor distancia**.

La consulta deberá evitar ciclos.

10.2. Grado de parentesco

Escribir una segunda consulta que determine el grado de parentesco entre dos personas.

Para este ejercicio se considerarán relaciones familiares directas: padre/madre–hijo/hija, hermanos/as y cónyuges.

El grado de parentesco será la **mínima cantidad de relaciones familiares directas** que deben recorrerse para conectar ambas personas.

Por ejemplo:

- relación familiar directa → grado 1;
- dos relaciones directas → grado 2;
- tres relaciones directas → grado 3.

Si no existe ninguna relación familiar entre las personas, el resultado deberá ser 0.

La consulta deberá contemplar la existencia de ciclos y de múltiples caminos posibles.

11. Descomposición de productos en partes

Crear, si no estuviera definida, una estructura que represente la composición de productos mediante una tabla:

```
partes(idProducto, idParte, cantidad)
```

Cada parte podrá ser, a su vez, un producto compuesto por otras partes. Por lo tanto, la estructura puede tener una cantidad arbitraria de niveles.

Por ejemplo, un automóvil puede contener ruedas; una rueda puede contener una llanta y un neumático; y determinados componentes pueden, a su vez, estar formados por otros componentes.

Escribir una **función SQL** que, dado un `idProducto`, devuelva todas las partes y subpartes que lo componen.

Para cada elemento deberá indicar como mínimo:

- `idProducto`;

- `idParte`;
- nivel en el que se encuentra;
- cantidad requerida.

El cuerpo de la función deberá estar escrito en SQL y utilizar `WITH RECURSIVE`.

Extender la solución anterior para calcular la cantidad real de cada componente necesaria para fabricar **una unidad del producto original**.

Por ejemplo, si un producto requiere 4 unidades de una parte A y cada unidad de A requiere 3 unidades de B, deberán considerarse necesarias 12 unidades de B para fabricar el producto original.

12. Participación porcentual de las partes

A partir de la tabla `partes`, escribir una consulta que devuelva:

- `idProducto`;
- `idParte`;
- `cantidad`;
- porcentaje que representa dicha cantidad respecto del total de partes directas requeridas por el producto.

Por ejemplo, si para un producto la suma de las cantidades de todas sus partes es 25 y una determinada parte tiene una cantidad igual a 5, su participación será del 20 %.

Resolver el ejercicio utilizando una **función de ventana**, evitando realizar una consulta independiente para calcular el total de cada producto.

13. Ranking de partes por producto

Utilizando la misma tabla `partes` y funciones de ventana, escribir una consulta que muestre, para cada producto:

- `idProducto`;
- `idParte`;
- `cantidad`;
- porcentaje sobre el total de partes del producto;
- posición de la parte según su cantidad, de mayor a menor.

En caso de que dos partes tengan la misma cantidad, deberán ocupar la misma posición en el ranking.

Analice cuál de las funciones `ROW_NUMBER()`, `RANK()` o `DENSE_RANK()` resulta más apropiada para cumplir este requisito.

Consideraciones generales

En todos los ejercicios se deberá:

- utilizar nombres significativos para tablas, funciones y triggers;
- respetar las claves primarias y foráneas necesarias para garantizar la integridad referencial;
- evitar la duplicación innecesaria de lógica;
- considerar valores nulos cuando corresponda;
- evitar ciclos infinitos en todas las consultas recursivas;
- probar las soluciones con un conjunto de datos suficientemente representativo.

Cuando un ejercicio solicite explícitamente la utilización de `WITH`, `WITH RECURSIVE`, funciones, triggers o funciones de ventana, la solución deberá emplear el mecanismo indicado.

Además del código SQL, se deberán incluir sentencias de prueba que permitan verificar el correcto funcionamiento de las soluciones implementadas.